```
"""
╔═══════════════════════════════════════════╗
║     NONENAME CHAT  v2.2                    ║
║     Referral + Confirm + Admin + Status    ║
╚═══════════════════════════════════════════╝
"""

import telebot
from telebot import types
import time, threading, logging, os, uuid, sqlite3, datetime as _dt
from datetime import datetime
from typing import Optional, List

# ═══════════════════════════════════════════
#  ⚙️  НАСТРОЙКИ
# ═══════════════════════════════════════════

BOT_TOKEN     = "8797966321:AAHyLfOLsIH5u2fwFkUDkaB57cWn4CJ7Fys"
ADMIN_IDS     = [5618560527]
CREATOR_ID    = 5618560527    # ID создателя
MODERATOR_IDS = [5618560527]  # список модераторов
BOT_USERNAME  = "NoneName_Chat_Bot"
DB_FILE       = "nonename.db"
RATE_LIMIT    = 0.7
MEDIA_PER_MIN = 3
REFS_FOR_VIP  = 3
VIP_DAYS_REF  = 7

BASE_DIR   = os.path.dirname(os.path.abspath(__file__))
IMAGES_DIR = os.path.join(BASE_DIR, "images")
HEART_GIF  = os.path.join(IMAGES_DIR, "heart.gif")

STARS_TO_NAYMS = 10
HEART_PRICE    = 30
HEART_SELL_OWN = 24
HEART_SELL_GIFT= 24
DM_LIMIT_DAY   = 10

# ═══════════════════════════════════════════
#  📋  КОНСТАНТЫ
# ═══════════════════════════════════════════

INTERESTS = [
    "💬 Просто поговорить","🎮 Игры","🎵 Музыка",
```

```
    "📚 Книги / Кино","🐱 Животные","✈️ Путешествия",
]
MOODS = ["😊 Хочу поговорить","🔥 Хочу поспорить","😴 Просто скучно","🎲 Без нас
REPORT_REASONS = ["🔞 Пошлый контент","📢 Реклама / Спам","🤬 Неадекватный","🤖
PROFILE_STEPS  = ["city","age","gender","about"]
PROFILE_Q = {
    "city":   "Введите ваш <b>город</b>:",
    "age":    "Введите ваш <b>возраст</b>:",
    "gender": "Введите ваш <b>пол</b> (мужской / женский / другой):",
    "about":  "Расскажите <b>о себе</b> (пару слов):",
}
RANKS = [
    (0,   "🐣 Новичок"),
    (100, "🏙️ Местный"),
    (200, "📜 Знаток"),
    (300, "🥷 Магистр"),
    (500, "🔥 Легенда чата"),
]
ALL_MENU = [
    "🔍 Найти собеседника","👑 VIP-чат","⭐ Купить VIP",
    "📋 Моя анкета","📜 История чатов","ℹ️ Мой профиль",
    "❌ Завершить диалог","🚩 Пожаловаться","👤 Анкета собеседника",
    "⏹️ Покинуть очередь","🌍 Поиск по городу","⏩ Без фильтра",
    "⏭️ Пропустить","↩️ Отмена","🔙 Выйти из панели",
    "🔨 Забанить","✅ Разбанить","👑 Выдать VIP","❌ Снять VIP",
    "📊 Статистика","📋 Жалобы","📣 Рассылка","🔗 Реферальная система",
    "🔄 Авто-следующий: ВКЛ","🔄 Авто-следующий: ВЫКЛ",
    "📅 Ежедневный бонус",
    "✉️ Написать пользователю","🔗 Рефералы (admин)",
    "⏭️ Пропустить сообщение",
    "💰 Кошелёк","🛍️ Магазин","🎁 Мои подарки","🎁 Подарок",
    "🔎 Поиск людей",
    "✅ Да, добавить записку","⏭️ Нет, без записки",
    "📩 Анонимно","📨 Под своим ником",
    "📢 Тексты для пиара",
    "🔄 Скрыть статус","🔄 Показать статус",
] + INTERESTS + MOODS + REPORT_REASONS

# ═══════════════════════════════════════════
#  🗄️  БАЗА ДАННЫХ
# ═══════════════════════════════════════════

def get_conn():
    conn = sqlite3.connect(DB_FILE, check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA journal_mode=WAL")
    return conn
```

```
def init_db():
    conn = get_conn()
    c = conn.cursor()

    c.execute("""
    CREATE TABLE IF NOT EXISTS users (
        uid           INTEGER PRIMARY KEY,
        username      TEXT    DEFAULT '',
        nayms         INTEGER DEFAULT 0,
        chats         INTEGER DEFAULT 0,
        msgs          INTEGER DEFAULT 0,
        streak        INTEGER DEFAULT 0,
        last_visit    TEXT    DEFAULT '',
        bonus_today   INTEGER DEFAULT 0,
        vip_until     REAL    DEFAULT 0,
        banned        INTEGER DEFAULT 0,
        ban_reason    TEXT    DEFAULT '',
        ban_until     REAL    DEFAULT 0,
        ref_code      TEXT    DEFAULT '',
        ref_claimed   INTEGER DEFAULT 0,
        created_at    TEXT    DEFAULT '',
        dm_sent_today INTEGER DEFAULT 0,
        dm_date       TEXT    DEFAULT '',
        incognito_mode INTEGER DEFAULT 0
    )""")

    existing = [row[1] for row in c.execute("PRAGMA table_info(users)")]
    migrations = {
        "nayms":          "INTEGER DEFAULT 0",
        "dm_sent_today":  "INTEGER DEFAULT 0",
        "dm_date":        "TEXT DEFAULT ''",
        "ref_code":       "TEXT DEFAULT ''",
        "ref_claimed":    "INTEGER DEFAULT 0",
        "banned":         "INTEGER DEFAULT 0",
        "ban_reason":     "TEXT DEFAULT ''",
        "ban_until":      "REAL DEFAULT 0",
        "vip_until":      "REAL DEFAULT 0",
        "bonus_today":    "INTEGER DEFAULT 0",
        "last_visit":     "TEXT DEFAULT ''",
        "streak":         "INTEGER DEFAULT 0",
        "username":       "TEXT DEFAULT ''",
        "msgs":           "INTEGER DEFAULT 0",
        "chats":          "INTEGER DEFAULT 0",
        "created_at":     "TEXT DEFAULT ''",
        "incognito_mode": "INTEGER DEFAULT 0",
    }
```

```
    for col, dtype in migrations.items():
        if col not in existing:
            try:
                c.execute(f"ALTER TABLE users ADD COLUMN {col} {dtype}")
                logging.info(f"Миграция: добавлена колонка users.{col}")
            except Exception as e:
                logging.warning(f"Миграция users.{col}: {e}")

    c.execute("""CREATE TABLE IF NOT EXISTS refs (
        inviter_id INTEGER, invited_id INTEGER,
        PRIMARY KEY (inviter_id, invited_id))""")

    c.execute("""CREATE TABLE IF NOT EXISTS inventory (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER, item_name TEXT, received_at TEXT,
        is_purchased INTEGER DEFAULT 1,
        FOREIGN KEY(user_id) REFERENCES users(uid))""")

    c.execute("""CREATE TABLE IF NOT EXISTS reports (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        reporter_id INTEGER, target_id INTEGER,
        reason TEXT, created_at TEXT)""")

    c.execute("""CREATE TABLE IF NOT EXISTS profiles (
        uid INTEGER PRIMARY KEY,
        city TEXT DEFAULT '', age TEXT DEFAULT '',
        gender TEXT DEFAULT '', about TEXT DEFAULT '')""")

    c.execute("""CREATE TABLE IF NOT EXISTS history (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        pair_key TEXT, sender TEXT, text TEXT, created_at TEXT)""")

    c.execute("""CREATE TABLE IF NOT EXISTS messages (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        from_uid INTEGER, to_uid INTEGER, text TEXT,
        is_anon INTEGER DEFAULT 0, created_at TEXT,
        is_read INTEGER DEFAULT 0)""")

    conn.commit(); conn.close()
    logging.info("БД готова (v2.2 migration OK).")

# ═══════════════════════════════════════════
# 💾 ХРАНИЛИЩЕ
# ═══════════════════════════════════════════

class Storage:
    def __init__(self):
```

```
        self._lock        = threading.Lock()
        self._chats       = {}
        self._queue       = []
        self._vip_queue   = []
        self._interests   = {}
        self._city_filter = {}
        self._mood        = {}
        self._last_msg    = {}
        self._media_times = {}
        self._pstate      = {}
        self._astate      = {}
        self._auto_next   = set()
        self._tmp_profile = {}
        self._db          = get_conn()

    def _q(self, sql, params=()):
        with self._lock:
            c = self._db.cursor()
            c.execute(sql, params)
            self._db.commit()
            return c

    def _fetch(self, sql, params=()):
        with self._lock:
            c = self._db.cursor()
            c.execute(sql, params)
            return c.fetchall()

    def _fetchone(self, sql, params=()):
        with self._lock:
            c = self._db.cursor()
            c.execute(sql, params)
            return c.fetchone()

    def register(self, uid:int, username:str=""):
        now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        code = uuid.uuid4().hex[:8]
        self._q("INSERT OR IGNORE INTO users(uid,username,created_at,ref_code) VA
                (uid, username, now, code))
        if username:
            self._q("UPDATE users SET username=? WHERE uid=?", (username, uid))

    def get_user(self, uid:int):
        return self._fetchone("SELECT * FROM users WHERE uid=?", (uid,))

    def all_users(self) -> List[int]:
        return [r["uid"] for r in self._fetch("SELECT uid FROM users")]
```

```
def get_username(self, uid:int) -> str:
    r = self._fetchone("SELECT username FROM users WHERE uid=?", (uid,))
    return r["username"] if r else ""

def find_by_username(self, username:str) -> Optional[int]:
    username = username.lstrip("@")
    r = self._fetchone("SELECT uid FROM users WHERE username=?", (username,))
    return r["uid"] if r else None

def search_users(self, query:str) -> list:
    query = query.strip()
    if query.startswith("@"): query = query[1:]
    results = []
    rows = self._fetch("SELECT uid,username FROM users WHERE username LIKE ?
                       (f"%{query}%",))
    results.extend(rows)
    if query.isdigit():
        r = self._fetchone("SELECT uid,username FROM users WHERE uid=?", (int
        if r and r not in results: results.append(r)
    return results

def get_nayms(self, uid:int) -> int:
    r = self._fetchone("SELECT nayms FROM users WHERE uid=?", (uid,))
    return r["nayms"] if r else 0

def add_nayms(self, uid:int, amount:int):
    self._q("UPDATE users SET nayms=MAX(0,nayms+?) WHERE uid=?", (amount, uid

def has_nayms(self, uid:int, amount:int) -> bool:
    return self.get_nayms(uid) >= amount

def give_vip(self, uid:int, days:int):
    r = self._fetchone("SELECT vip_until FROM users WHERE uid=?", (uid,))
    now = time.time()
    cur = max(r["vip_until"] if r else now, now)
    self._q("UPDATE users SET vip_until=? WHERE uid=?", (cur+days*86400, uid)

def remove_vip(self, uid:int):
    self._q("UPDATE users SET vip_until=0 WHERE uid=?", (uid,))

def is_vip(self, uid:int) -> bool:
    r = self._fetchone("SELECT vip_until FROM users WHERE uid=?", (uid,))
    return bool(r and r["vip_until"] > time.time())

def vip_expires(self, uid:int) -> Optional[str]:
    r = self._fetchone("SELECT vip_until FROM users WHERE uid=?", (uid,))
```

```
    if r and r["vip_until"] > time.time():
        return datetime.fromtimestamp(r["vip_until"]).strftime("%d.%m.%Y %H:%
    return None

def ban(self, uid:int, reason:str, days:int=0) -> Optional[int]:
    until = time.time()+days*86400 if days>0 else 0
    self._q("UPDATE users SET banned=1,ban_reason=?,ban_until=? WHERE uid=?",
            (reason, until, uid))
    with self._lock:
        self._queue_rm(uid); self._vqueue_rm(uid)
        p = self._chats.pop(uid, None)
        if p: self._chats.pop(p, None)
    return p

def unban(self, uid:int):
    self._q("UPDATE users SET banned=0,ban_reason='',ban_until=0 WHERE uid=?"

def is_banned(self, uid:int) -> bool:
    r = self._fetchone("SELECT banned,ban_until FROM users WHERE uid=?", (uid
    if not r or not r["banned"]: return False
    if r["ban_until"]>0 and time.time()>r["ban_until"]:
        self.unban(uid); return False
    return True

def ban_info(self, uid:int):
    return self._fetchone("SELECT ban_reason,ban_until FROM users WHERE uid=?

def create_chat(self, u1:int, u2:int):
    with self._lock:
        self._chats[u1]=u2; self._chats[u2]=u1
        self._queue_rm(u1); self._queue_rm(u2)
        self._vqueue_rm(u1); self._vqueue_rm(u2)
    self._q("UPDATE users SET chats=chats+1 WHERE uid IN (?,?)", (u1, u2))

def end_chat(self, uid:int) -> Optional[int]:
    with self._lock:
        p = self._chats.pop(uid, None)
        if p: self._chats.pop(p, None)
        self._queue_rm(uid); self._vqueue_rm(uid)
        return p

def get_partner(self, uid:int) -> Optional[int]:
    with self._lock: return self._chats.get(uid)

def is_in_chat(self, uid:int) -> bool:
    with self._lock: return uid in self._chats
```

```
def add_queue(self, uid:int, vip:bool=False):
    with self._lock:
        if vip:
            if uid not in self._vip_queue: self._vip_queue.append(uid)
        else:
            if uid not in self._queue: self._queue.append(uid)

def leave_queue(self, uid:int):
    with self._lock: self._queue_rm(uid); self._vqueue_rm(uid)

def _queue_rm(self, uid):
    if uid in self._queue: self._queue.remove(uid)

def _vqueue_rm(self, uid):
    if uid in self._vip_queue: self._vip_queue.remove(uid)

def in_queue(self, uid:int) -> bool:
    with self._lock: return uid in self._queue or uid in self._vip_queue

def in_vip_queue(self, uid:int) -> bool:
    with self._lock: return uid in self._vip_queue

def q_snap(self) -> List[int]:
    with self._lock: return list(self._queue)

def vq_snap(self) -> List[int]:
    with self._lock: return list(self._vip_queue)

def find_match(self, uid:int, vip:bool=False) -> Optional[int]:
    with self._lock:
        pool = self._vip_queue if vip else self._queue
        cands = [u for u in pool if u!=uid]
        if not cands: return None
        my_city = self._city_filter.get(uid) if vip else None
        my_int  = self._interests.get(uid)
        my_mood = self._mood.get(uid)
        if my_city and my_int:
            for u in cands:
                if self._city_filter.get(u)==my_city and self._interests.get(
        if my_city:
            for u in cands:
                if self._city_filter.get(u)==my_city: return u
        if my_mood:
            for u in cands:
                if self._mood.get(u)==my_mood: return u
        if my_int:
            for u in cands:
```

```
                if self._interests.get(u)==my_int: return u
        for u in cands:
            if not self._interests.get(u): return u
        return cands[0]

    def set_interest(self, uid, v): self._interests[uid]=v
    def set_city(self, uid, v):     self._city_filter[uid]=v
    def set_mood(self, uid, v):     self._mood[uid]=v

    def rate_limited(self, uid:int) -> bool:
        with self._lock:
            return (time.time()-self._last_msg.get(uid,0)) < RATE_LIMIT

    def record_msg(self, uid:int):
        with self._lock: self._last_msg[uid]=time.time()
        self._q("UPDATE users SET msgs=msgs+1 WHERE uid=?", (uid,))

    def media_limited(self, uid:int) -> bool:
        now = time.time()
        with self._lock:
            times = [t for t in self._media_times.get(uid,[]) if now-t<60]
            self._media_times[uid] = times
            if len(times)>=MEDIA_PER_MIN: return True
            self._media_times[uid].append(now); return False

    def can_send_dm(self, uid:int) -> bool:
        if self.is_vip(uid): return True
        today = _dt.date.today().strftime("%Y-%m-%d")
        r = self._fetchone("SELECT dm_sent_today,dm_date FROM users WHERE uid=?",
        if not r: return True
        if r["dm_date"] != today: return True
        return r["dm_sent_today"] < DM_LIMIT_DAY

    def record_dm_sent(self, uid:int):
        today = _dt.date.today().strftime("%Y-%m-%d")
        r = self._fetchone("SELECT dm_date FROM users WHERE uid=?", (uid,))
        if r and r["dm_date"] == today:
            self._q("UPDATE users SET dm_sent_today=dm_sent_today+1 WHERE uid=?",
        else:
            self._q("UPDATE users SET dm_sent_today=1,dm_date=? WHERE uid=?", (to

    def dm_left(self, uid:int) -> int:
        if self.is_vip(uid): return 999
        today = _dt.date.today().strftime("%Y-%m-%d")
        r = self._fetchone("SELECT dm_sent_today,dm_date FROM users WHERE uid=?",
        if not r or r["dm_date"]!=today: return DM_LIMIT_DAY
        return max(0, DM_LIMIT_DAY - r["dm_sent_today"])
```

```
def add_report(self, target:int, reporter:int, reason:str) -> int:
    now = datetime.now().strftime("%d.%m %H:%M")
    self._q("INSERT INTO reports(reporter_id,target_id,reason,created_at) VAL
            (reporter, target, reason, now))
    r = self._fetchone("SELECT COUNT(*) as cnt FROM reports WHERE target_id=?
    return r["cnt"] if r else 0

def report_count(self, uid:int) -> int:
    r = self._fetchone("SELECT COUNT(*) as cnt FROM reports WHERE target_id=?
    return r["cnt"] if r else 0

def get_reports(self, uid:int):
    return self._fetch("SELECT * FROM reports WHERE target_id=? ORDER BY id D

def all_reported_users(self):
    return self._fetch("SELECT target_id,COUNT(*) as cnt FROM reports GROUP B

def save_msg_history(self, u1:int, u2:int, sender:int, text:str):
    key = f"{min(u1,u2)}_{max(u1,u2)}"
    frm = "вы" if sender==u1 else "собеседник"
    now = datetime.now().strftime("%H:%M")
    self._q("INSERT INTO history(pair_key,sender,text,created_at) VALUES(?,?,
            (key, frm, text[:200], now))
    self._q("DELETE FROM history WHERE pair_key=? AND id NOT IN (SELECT id FR
            (key, key))

def get_history(self, u1:int, u2:int):
    key = f"{min(u1,u2)}_{max(u1,u2)}"
    return self._fetch("SELECT * FROM history WHERE pair_key=? ORDER BY id DE

def set_profile(self, uid:int, p:dict):
    self._q("INSERT OR REPLACE INTO profiles(uid,city,age,gender,about) VALUE
            (uid, p.get("city",""), p.get("age",""), p.get("gender",""), p.ge

def get_profile(self, uid:int):
    return self._fetchone("SELECT * FROM profiles WHERE uid=?", (uid,))

def del_profile(self, uid:int):
    self._q("DELETE FROM profiles WHERE uid=?", (uid,))

def get_ref_code(self, uid:int) -> str:
    r = self._fetchone("SELECT ref_code FROM users WHERE uid=?", (uid,))
    if r and r["ref_code"]: return r["ref_code"]
    code = uuid.uuid4().hex[:8]
    self._q("UPDATE users SET ref_code=? WHERE uid=?", (code, uid))
    return code
```

```
    def use_ref(self, code:str, new_uid:int):
        r = self._fetchone("SELECT uid FROM users WHERE ref_code=?", (code,))
        if not r or r["uid"]==new_uid: return None
        inviter = r["uid"]
        ex = self._fetchone("SELECT 1 FROM refs WHERE inviter_id=? AND invited_id
        if ex: return None
        self._q("INSERT INTO refs(inviter_id,invited_id) VALUES(?,?)", (inviter,
        cnt = self._fetchone("SELECT COUNT(*) as c FROM refs WHERE inviter_id=?",
        return inviter, cnt["c"]

    def ref_count(self, uid:int) -> int:
        r = self._fetchone("SELECT COUNT(*) as c FROM refs WHERE inviter_id=?", (
        return r["c"] if r else 0

    def ref_claimed(self, uid:int) -> bool:
        r = self._fetchone("SELECT ref_claimed FROM users WHERE uid=?", (uid,))
        return bool(r and r["ref_claimed"])

    def mark_ref_claimed(self, uid:int):
        self._q("UPDATE users SET ref_claimed=1 WHERE uid=?", (uid,))

    def all_refs(self):
        return self._fetch("SELECT inviter_id,COUNT(*) as cnt FROM refs GROUP BY

    def toggle_auto_next(self, uid:int) -> bool:
        with self._lock:
            if uid in self._auto_next: self._auto_next.discard(uid); return False
            else: self._auto_next.add(uid); return True

    def is_auto_next(self, uid:int) -> bool:
        with self._lock: return uid in self._auto_next

    def get_streak(self, uid:int) -> int:
        r = self._fetchone("SELECT streak FROM users WHERE uid=?", (uid,))
        return r["streak"] if r else 0

    def record_daily_visit(self, uid:int):
        today = _dt.date.today().strftime("%Y-%m-%d")
        prev  = (_dt.date.today()-_dt.timedelta(days=1)).strftime("%Y-%m-%d")
        r = self._fetchone("SELECT streak,last_visit,bonus_today FROM users WHERE
        if not r: return 0, False
        streak=r["streak"]; last=r["last_visit"]; bonus=r["bonus_today"]
        if last==today: return streak, bool(bonus)
        streak=(streak+1) if last==prev else 1
        gave=(streak>=7 and not bonus)
        self._q("UPDATE users SET streak=?,last_visit=?,bonus_today=? WHERE uid=?
```

```
            (streak, today, 1 if gave else 0, uid))
    return streak, gave

def set_ps(self,uid,s):  self._pstate[uid]=s
def get_ps(self,uid):    return self._pstate.get(uid)
def clr_ps(self,uid):    self._pstate.pop(uid,None)
def set_as(self,uid,d):  self._astate[uid]=d
def get_as(self,uid):    return self._astate.get(uid)
def clr_as(self,uid):    self._astate.pop(uid,None)

def add_item(self, uid:int, item_name:str, is_purchased:int=1):
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    self._q("INSERT INTO inventory(user_id,item_name,received_at,is_purchased
            (uid, item_name, now, is_purchased))

def get_inventory(self, uid:int):
    return self._fetch("SELECT * FROM inventory WHERE user_id=? ORDER BY id D

def get_item(self, item_id:int, uid:int):
    return self._fetchone("SELECT * FROM inventory WHERE id=? AND user_id=?",

def remove_item(self, item_id:int):
    self._q("DELETE FROM inventory WHERE id=?", (item_id,))

def count_item(self, uid:int, item_name:str) -> int:
    r = self._fetchone("SELECT COUNT(*) as c FROM inventory WHERE user_id=? A
                       (uid, item_name))
    return r["c"] if r else 0

def save_dm(self, from_uid:int, to_uid:int, text:str, is_anon:int=0):
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    self._q("INSERT INTO messages(from_uid,to_uid,text,is_anon,created_at) VA
            (from_uid, to_uid, text, is_anon, now))

def get_dm_history(self, uid1:int, uid2:int):
    return self._fetch("""SELECT * FROM messages
        WHERE (from_uid=? AND to_uid=?) OR (from_uid=? AND to_uid=?)
        ORDER BY id DESC LIMIT 20""", (uid1, uid2, uid2, uid1))

# ── v2.2: инкогнито ────────────────────────────────────────

def is_incognito(self, uid:int) -> bool:
    r = self._fetchone("SELECT incognito_mode FROM users WHERE uid=?", (uid,)
    return bool(r and r["incognito_mode"])

def toggle_incognito(self, uid:int) -> bool:
    cur = self.is_incognito(uid)
```

```
        self._q("UPDATE users SET incognito_mode=? WHERE uid=?", (0 if cur else 1
        return not cur


# ═══════════════════════════════════════════════
#  🏆  ГЕЙМИФИКАЦИЯ
# ═══════════════════════════════════════════════

def get_rank(chats:int) -> str:
    rank = RANKS[0][1]
    for t,title in RANKS:
        if chats>=t: rank=title
    return rank

def get_next_rank(chats:int):
    for t,title in RANKS:
        if chats<t: return title, t-chats
    return None, 0

# ═══════════════════════════════════════════════
#  🎹  КЛАВИАТУРЫ
# ═══════════════════════════════════════════════

def kb_main(uid:int) -> types.ReplyKeyboardMarkup:
    vip = storage.is_vip(uid)
    kb  = types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    kb.add(types.KeyboardButton("🔍 Найти собеседника"))
    if vip: kb.add(types.KeyboardButton("👑 VIP-чат"))
    kb.add(types.KeyboardButton("⭐ Купить VIP"), types.KeyboardButton("🔗 Рефера
    kb.add(types.KeyboardButton("💰 Кошелёк"),    types.KeyboardButton("🛍️ Магази
    kb.add(types.KeyboardButton("🎁 Мои подарки"),types.KeyboardButton("🔎 Поиск
    if vip:
        an = "🔄 Авто-следующий: ВКЛ" if storage.is_auto_next(uid) else "🔄 Авто-
        kb.add(types.KeyboardButton("📋 Моя анкета"), types.KeyboardButton("📜 Ис
        kb.add(types.KeyboardButton(an))
    kb.add(types.KeyboardButton("📅 Ежедневный бонус"), types.KeyboardButton("ℹ️
    return kb

def kb_chat(uid:int) -> types.ReplyKeyboardMarkup:
    kb = types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    kb.add(types.KeyboardButton("❌ Завершить диалог"), types.KeyboardButton("🚩
    kb.add(types.KeyboardButton("🎁 Подарок"))
    if storage.is_vip(uid) and storage.get_partner(uid):
        kb.add(types.KeyboardButton("👤 Анкета собеседника"))
    return kb

def kb_interests() -> types.ReplyKeyboardMarkup:
```

```
    kb = types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    for i in INTERESTS: kb.add(types.KeyboardButton(i))
    kb.add(types.KeyboardButton("⏩ Пропустить"))
    return kb

def kb_mood() -> types.ReplyKeyboardMarkup:
    kb = types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    for m in MOODS: kb.add(types.KeyboardButton(m))
    kb.add(types.KeyboardButton("⏩ Пропустить"))
    return kb

def kb_report() -> types.ReplyKeyboardMarkup:
    kb = types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=1)
    for r in REPORT_REASONS: kb.add(types.KeyboardButton(r))
    kb.add(types.KeyboardButton("↩️ Отмена"))
    return kb

def kb_vip_buy() -> types.InlineKeyboardMarkup:
    kb = types.InlineKeyboardMarkup(row_width=1)
    kb.add(
        types.InlineKeyboardButton("VIP на 7 дней — 5 ⭐",  callback_data="buy_vi
        types.InlineKeyboardButton("VIP на 30 дней — 10 ⭐", callback_data="buy_v
    )
    return kb

def kb_wallet() -> types.InlineKeyboardMarkup:
    kb = types.InlineKeyboardMarkup(row_width=1)
    kb.add(
        types.InlineKeyboardButton("100 ⚡ — 10 ⭐",  callback_data="buy_nayms_1(
        types.InlineKeyboardButton("250 ⚡ — 25 ⭐",  callback_data="buy_nayms_25
        types.InlineKeyboardButton("500 ⚡ — 50 ⭐",  callback_data="buy_nayms_5(
    )
    return kb

def kb_shop(uid:int) -> types.InlineKeyboardMarkup:
    kb = types.InlineKeyboardMarkup(row_width=1)
    nayms = storage.get_nayms(uid)
    label = f"💝 Купить Сердце — {HEART_PRICE} ⚡"
    if nayms < HEART_PRICE:
        label += f" (не хватает {HEART_PRICE-nayms} ⚡)"
    kb.add(types.InlineKeyboardButton(label, callback_data="buy_item_heart"))
    return kb

def kb_gift_select() -> types.InlineKeyboardMarkup:
    kb = types.InlineKeyboardMarkup(row_width=1)
    kb.add(types.InlineKeyboardButton("💝 Подарить Сердце", callback_data="gift_h
    return kb
```

```
def kb_vip_search() -> types.ReplyKeyboardMarkup:
    kb = types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    kb.add(types.KeyboardButton("🌍 Поиск по городу"))
    for i in INTERESTS: kb.add(types.KeyboardButton(i))
    kb.add(types.KeyboardButton("⏩ Без фильтра"))
    return kb

def kb_leave() -> types.ReplyKeyboardMarkup:
    kb = types.ReplyKeyboardMarkup(resize_keyboard=True)
    kb.add(types.KeyboardButton("⏹ Покинуть очередь"))
    return kb

def kb_admin(uid:int=0) -> types.ReplyKeyboardMarkup:
    kb = types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    kb.add(
        types.KeyboardButton("🔨 Забанить"),           types.KeyboardButton("✅ Р
        types.KeyboardButton("👑 Выдать VIP"),          types.KeyboardButton("❌
        types.KeyboardButton("📊 Статистика"),          types.KeyboardButton("📋
        types.KeyboardButton("📤 Рассылка"),            types.KeyboardButton("✉
        types.KeyboardButton("🔗 Рефералы (админ)"),
    )
    # Кнопка статуса зависит от текущего режима
    if uid and storage.is_incognito(uid):
        kb.add(types.KeyboardButton("🔄 Показать статус"))
    else:
        kb.add(types.KeyboardButton("🔄 Скрыть статус"))
    kb.add(types.KeyboardButton("🔙 Выйти из панели"))
    return kb

def kb_public_profile(target_uid:int) -> types.InlineKeyboardMarkup:
    kb = types.InlineKeyboardMarkup(row_width=2)
    kb.add(
        types.InlineKeyboardButton("🎁 Подарить", callback_data=f"pgift_{target_u
        types.InlineKeyboardButton("✉ Написать", callback_data=f"pdm_{target_uid
    )
    return kb

# ═══════════════════════════════════════════════
#  🤖  ИНИЦИАЛИЗАЦИЯ
# ═══════════════════════════════════════════════

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.FileHandler("bot.log",encoding="utf-8"), logging.StreamHand
logger = logging.getLogger(__name__)
```

```
init_db()
bot     = telebot.TeleBot(BOT_TOKEN, parse_mode="HTML")
storage = Storage()

# ═══════════════════════════════════════════════
#  🛠️  УТИЛИТЫ
# ═══════════════════════════════════════════════

def send(cid, text, markup=None, **kw):
    try:   return bot.send_message(cid, text, reply_markup=markup, **kw)
    except Exception as e: logger.warning(f"send({cid}): {e}")

def is_admin(uid:int) -> bool: return uid in ADMIN_IDS

def fmt_num(n:int) -> str: return f"{n:,}".replace(",", " ")

def fmt_profile(p, public=False) -> str:
    if not p: return "Анкета не заполнена."
    lines = ["👤 <b>Анкета пользователя:</b>"]
    if p["city"]:   lines.append(f"🏙️ Город: {p['city']}")
    if p["age"]:    lines.append(f"🎂 Возраст: {p['age']}")
    if p["gender"]: lines.append(f"⚧️ Пол: {p['gender']}")
    if p["about"]:  lines.append(f"📝 О себе: {p['about']}")
    return "\n".join(lines)

def partner_left_msg(pid:int):
    send(pid, "❌ <b>Собеседник завершил диалог.</b>\n\nНажмите «Найти собеседник
         markup=kb_main(pid))

def update_bot_description():
    try:
        num = fmt_num(len(storage.all_users()))
        bot.set_my_description(
            f"NoneName Chat — твоя зона инкогнито. Нас уже {num}! "
            f"Находи друзей, общайся и оставайся в тени.")
        bot.set_my_short_description(
            f"Твоя зона инкогнито 🥷 Нас уже {num} в тени!")
    except Exception as e: logger.warning(f"desc: {e}")

def count_active_users() -> int:
    active = 0
    for uid in storage.all_users():
        try: bot.send_chat_action(uid,"typing"); active+=1
        except: pass
    return active
```

```
def resolve_target(text:str) -> Optional[int]:
    """Принимает ID или @username, возвращает uid."""
    text = text.strip()
    if text.isdigit(): return int(text)
    return storage.find_by_username(text)

def do_match(uid:int, vip:bool=False) -> bool:
    pid = storage.find_match(uid, vip=vip)
    if pid is None: return False
    storage.create_chat(uid, pid)
    uid_vip=storage.is_vip(uid); pid_vip=storage.is_vip(pid)
    def ft(ov): return ("✅ Найден <b>VIP-пользователь! 👑</b>" if ov
                        else "✅ <b>Собеседник найден! 😊</b>") + \
                       "\n\nВы анонимно подключены. Пишите — сообщения передаются
    send(uid, ft(pid_vip), markup=kb_chat(uid))
    send(pid, ft(uid_vip), markup=kb_chat(pid))
    if uid_vip and pid_vip:
        p1,p2=storage.get_profile(uid),storage.get_profile(pid)
        if p2: send(uid, fmt_profile(p2))
        if p1: send(pid, fmt_profile(p1))
    logger.info(f"Чат: {uid}<->{pid} vip={vip}")
    return True

def fwd(msg, to_id:int):
    """Пересылка с учётом статуса создателя/модератора."""
    uid = msg.from_user.id
    ct  = msg.content_type
    cap = msg.caption

    # Определяем префикс статуса
    prefix = ""
    if not storage.is_incognito(uid):
        if uid == CREATOR_ID:
            prefix = "[💻 Создатель] "
        elif uid in MODERATOR_IDS:
            prefix = "[🛡️ Модератор] "

    try:
        if ct == "text":
            text = (prefix + msg.text) if prefix else msg.text
            bot.send_message(to_id, text)
        elif ct == "photo":
            c = (prefix + cap) if (prefix and cap) else (cap or prefix or None)
            bot.send_photo(to_id, msg.photo[-1].file_id, caption=c)
        elif ct == "video":
            c = (prefix + cap) if (prefix and cap) else (cap or prefix or None)
            bot.send_video(to_id, msg.video.file_id, caption=c)
```

```
        elif ct == "audio":
            bot.send_audio(to_id, msg.audio.file_id, caption=cap)
        elif ct == "voice":
            bot.send_voice(to_id, msg.voice.file_id)
        elif ct == "video_note":
            bot.send_video_note(to_id, msg.video_note.file_id)
        elif ct == "sticker":
            bot.send_sticker(to_id, msg.sticker.file_id)
        elif ct == "document":
            bot.send_document(to_id, msg.document.file_id, caption=cap)
        elif ct == "animation":
            bot.send_animation(to_id, msg.animation.file_id, caption=cap)
        else:
            bot.send_message(to_id, "[Неподдерживаемый тип]")
        # Для медиа без caption — отправляем статус отдельно
        if prefix and ct not in ("text","photo","video"):
            bot.send_message(to_id, f"<i>{prefix.strip()}</i>")
    except telebot.apihelper.ApiTelegramException as e: raise e

def send_heart_animation(to_id:int, caption:str=""):
    try:
        if os.path.exists(HEART_GIF):
            with open(HEART_GIF,"rb") as f:
                bot.send_animation(to_id, f, caption=caption or "💝")
        else:
            send(to_id, f"💝 {caption}" if caption else "💝 Вам отправили подарок
    except Exception as e:
        logger.warning(f"send_heart: {e}")
        send(to_id, f"💝 {caption}" if caption else "💝 Вам подарили Сердце!")

# ═══════════════════════════════════════════════
# 📩 КОМАНДЫ
# ═══════════════════════════════════════════════

@bot.message_handler(commands=["start"])
def cmd_start(message):
    uid   = message.from_user.id
    args  = message.text.split()
    uname = message.from_user.username or ""
    storage.register(uid, uname)
    storage.end_chat(uid); storage.leave_queue(uid)
    storage.clr_ps(uid);   storage.clr_as(uid)

    if len(args)>1:
        result = storage.use_ref(args[1], uid)
        if result:
            inviter,cnt = result
```

```
            inv_u=storage.get_username(inviter); new_u=storage.get_username(uid)
            send(inviter,
                 f"🎉 По вашей ссылке зарегистрировался новый пользователь!\n"
                 f"Рефералов: <b>{cnt}/{REFS_FOR_VIP}</b>")
            for adm in ADMIN_IDS:
                try: bot.send_message(adm,
                    f"🔗 <b>Новый реферал!</b>\n"
                    f"Пригласил: <code>{inviter}</code>{' (@'+inv_u+')' if inv_u
                    f"Новый: <code>{uid}</code>{' (@'+new_u+')' if new_u else ''}
                    f"Всего: <b>{cnt}</b>")
                except: pass
            if cnt>=REFS_FOR_VIP and not storage.ref_claimed(inviter):
                storage.mark_ref_claimed(inviter); storage.give_vip(inviter, VIP_
                exp=storage.vip_expires(inviter)
                send(inviter, f"🎊 <b>Поздравляем!</b> {REFS_FOR_VIP} рефералов!\
                             f"👑 VIP на {VIP_DAYS_REF} дней до <b>{exp}</b>!")

    threading.Thread(target=update_bot_description, daemon=True).start()
    count    = fmt_num(len(storage.all_users()))
    vip_mark = " 👑" if storage.is_vip(uid) else ""
    send(uid,
         f"🕶️ <b>NoneName Chat — Твоя зона инкогнито{vip_mark}</b>\n\n"
         f"Пиши что хочешь, будь кем хочешь. Твоё имя и профиль скрыты от всех.\n
         f"📊 Ты стал <b>{count}</b> участником нашего сообщества!\n\n"
         f"👇 Нажми «Найти собеседника», чтобы исчезнуть в переписке.",
         markup=kb_main(uid))

@bot.message_handler(commands=["admin"])
def cmd_admin(message):
    if not is_admin(message.from_user.id): return
    send(message.from_user.id, "🔧 <b>Панель администратора</b>",
         markup=kb_admin(message.from_user.id))

@bot.message_handler(commands=["give_nayms"])
def cmd_give_nayms(message):
    if not is_admin(message.from_user.id): return
    parts=message.text.split()
    if len(parts)!=3: send(message.from_user.id,"Использование: /give_nayms [id]
    try:
        t,a=int(parts[1]),int(parts[2]); storage.add_nayms(t,a); bal=storage.get_
        send(message.from_user.id,f"✅ Выдано {a} ⚡ пользователю {t}. Баланс: {b
        send(t,f"💸 Администратор начислил <b>{a} ⚡ Nayms</b>!\nБаланс: <b>{bal}
    except: send(message.from_user.id,"❌ Неверные параметры.")

@bot.message_handler(commands=["take_nayms"])
def cmd_take_nayms(message):
    if not is_admin(message.from_user.id): return
```

```
    parts=message.text.split()
    if len(parts)!=3: send(message.from_user.id,"Использование: /take_nayms [id]
    try:
        t,a=int(parts[1]),int(parts[2]); storage.add_nayms(t,-a); bal=storage.get
        send(message.from_user.id,f"✅ Списано {a} ⚡ у {t}. Баланс: {bal} ⚡")
        send(t,f"📉 Администратор списал <b>{a} ⚡ Nayms</b>.\nБаланс: <b>{bal}
    except: send(message.from_user.id,"❌ Неверные параметры.")

# ── Профиль ──────────────────────────────────────────────────

@bot.message_handler(func=lambda m: m.text=="ℹ️ Мой профиль")
def btn_my_info(message):
    uid=message.from_user.id; u=storage.get_user(uid)
    if not u: send(uid,"Напишите /start"); return
    exp=storage.vip_expires(uid); refs=storage.ref_count(uid)
    chats=u["chats"]; rank=get_rank(chats)
    nr,needed=get_next_rank(chats); streak=u["streak"]
    progress=f"До следующего ранга: ещё <b>{needed}</b> диалогов" if nr else "🏆
    dm_left=storage.dm_left(uid)
    lines=[
        "👤 <b>Ваш профиль</b>\n",
        f"🆔 ID: <code>{uid}</code>",
        f"🎖️ Ранг: <b>{rank}</b>",
        f"📈 {progress}",
        f"👑 VIP: {'до '+exp if exp else 'нет'}",
        f"⚡ Nayms: <b>{u['nayms']}</b>",
        f"📩 Сообщений сегодня: {'∞' if storage.is_vip(uid) else f'{DM_LIMIT_DAY-
        f"🚫 Статус: {'заблокирован' if storage.is_banned(uid) else 'активен'}",
        f"💬 Диалогов: {chats}",
        f"✉️ Сообщений: {u['msgs']}",
        f"🔗 Рефералов: {refs}/{REFS_FOR_VIP}",
        f"📅 Серия входов: {streak}/7 дней",
        f"🚩 Жалоб получено: {storage.report_count(uid)}",
    ]
    send(uid, "\n".join(lines), markup=kb_main(uid))

# ── Ежедневный бонус ─────────────────────────────────────────

@bot.message_handler(func=lambda m: m.text=="📅 Ежедневный бонус")
def btn_daily(message):
    uid=message.from_user.id; streak,gave=storage.record_daily_visit(uid)
    if gave:
        storage.give_vip(uid,1); exp=storage.vip_expires(uid)
        send(uid,f"🎉 <b>7 дней подряд!</b> VIP на 1 день!\n👑 До: {exp}\n\nПродо
            markup=kb_main(uid)); return
    left=7-streak; bar="🟩"*streak+"⬜"*(7-streak)
    send(uid,
```

```
        f"📅 <b>Ежедневный бонус</b>\n\n7 дней подряд = <b>VIP на 1 день</b>!\n\
        f"Серия: {bar}\n📓 Дней: <b>{streak}/7</b>\n"
        f"{'🎁 Ещё '+str(left)+' дн. до бонуса!' if left>0 else '✅ Бонус уже по
        f"<i>Пропустите день — серия сбросится.</i>", markup=kb_main(uid))

# ── Реферальная система v2.2 ──────────────────────────────

@bot.message_handler(func=lambda m: m.text=="🔗 Реферальная система")
def btn_ref(message):
    uid  = message.from_user.id
    code = storage.get_ref_code(uid)
    cnt  = storage.ref_count(uid)
    link = f"https://t.me/{BOT_USERNAME}?start={code}"
    # Прогресс до следующего VIP
    progress = cnt % REFS_FOR_VIP
    bar = "🟩"*progress + "⬜"*(REFS_FOR_VIP-progress)

    kb = types.InlineKeyboardMarkup(row_width=1)
    kb.add(types.InlineKeyboardButton("📣 Тексты для пиара", callback_data="ref_p

    send(uid,
        f"🔗 <b>Реферальная система</b>\n\n"
        f"Приглашайте друзей по своей ссылке.\n"
        f"За каждые <b>{REFS_FOR_VIP} приглашённых</b> — <b>VIP на {VIP_DAYS_REF
        f"📎 Ваша ссылка:\n<code>{link}</code>\n\n"
        f"📊 Прогресс до VIP: {bar} <b>{progress}/{REFS_FOR_VIP}</b>\n"
        f"👥 Всего приглашено за всё время: <b>{cnt}</b>\n\n"
        f"{'✅ VIP уже получен! Продолжайте приглашать!' if storage.ref_claimed(
        markup=kb)

@bot.callback_query_handler(func=lambda c: c.data=="ref_promo")
def cb_ref_promo(call):
    try: bot.answer_callback_query(call.id)
    except: pass
    send(call.from_user.id,
        "🚀 <b>Стань нашим амбассадором и получай бонусы!</b>\n\n"
        "Чтобы тебе было проще приглашать людей и копить на VIP, "
        "мы подготовили готовые рекламные шаблоны и список групп для пиара.\n\n"
        "<b>Всё просто:</b>\n"
        "• Переходи в наш канал по кнопке ниже.\n"
        "• Копируй любой текст и вставляй свою ссылку.\n"
        "• Рассылай в чаты и забирай награды!\n\n"
        "👇 Все инструкции и тексты здесь:\nhttps://t.me/+1_5UM-YnzAdiZTBi")

# ── Авто-следующий ────────────────────────────────────────

@bot.message_handler(func=lambda m: m.text in ["🔄 Авто-следующий: ВКЛ","🔄 Авто-
```

```
def btn_auto_next(message):
    uid=message.from_user.id
    if not storage.is_vip(uid): send(uid,"🚫 Авто-следующий — только VIP."); retu
    state=storage.toggle_auto_next(uid)
    send(uid,f"🔄 Авто-следующий <b>{'включён ✅' if state else 'выключен ❌'}</b
         markup=kb_main(uid))

# ── Поиск собеседника ──────────────────────────────────────

@bot.message_handler(func=lambda m: m.text=="🔍 Найти собеседника")
def btn_find(message):
    uid=message.from_user.id
    if storage.is_banned(uid):
        info=storage.ban_info(uid)
        send(uid,f"🚫 Вы заблокированы.\n📋 Причина: {info['ban_reason'] if info
    if storage.is_in_chat(uid): send(uid,"⚠️ Вы уже в чате.",markup=kb_chat(uid))
    if storage.in_queue(uid):   send(uid,"⏳ Вы уже в очереди."); return
    send(uid,"😊 Выберите <b>настроение</b>:",markup=kb_mood())

@bot.message_handler(func=lambda m: m.text in MOODS or m.text=="⏩ Пропустить")
def btn_mood_select(message):
    uid=message.from_user.id
    if storage.in_queue(uid) or storage.is_in_chat(uid): return
    if storage.is_vip(uid) and message.text=="⏩ Без фильтра": return
    mood=None if message.text=="⏩ Пропустить" else message.text
    if message.text in MOODS or message.text=="⏩ Пропустить":
        storage.set_mood(uid,mood)
        send(uid,"🎯 Выберите <b>интересы</b>:",markup=kb_interests())

@bot.message_handler(func=lambda m: m.text in INTERESTS)
def btn_interest(message):
    uid=message.from_user.id
    if storage.in_queue(uid) or storage.is_in_chat(uid): return
    storage.set_interest(uid,message.text); storage.add_queue(uid,vip=False)
    send(uid,f"⏳ Поиск по интересу <b>{message.text}</b>…",markup=kb_leave())
    if not do_match(uid): logger.info(f"{uid} в очереди ({message.text})")

@bot.message_handler(func=lambda m: m.text=="👑 VIP-чат")
def btn_vip_chat(message):
    uid=message.from_user.id
    if not storage.is_vip(uid): send(uid,"🚫 VIP-чат — только VIP."); return
    if storage.is_in_chat(uid): send(uid,"⚠️ Вы уже в чате.",markup=kb_chat(uid))
    send(uid,"👑 <b>VIP-чат</b>\n\nВыберите параметры:",markup=kb_vip_search())

@bot.message_handler(func=lambda m: m.text=="🌍 Поиск по городу")
def btn_city(message):
    uid=message.from_user.id
```

```
    if not storage.is_vip(uid): return
    send(uid,"🏙️ Введите название города:"); storage.set_ps(uid,"city_search")

@bot.message_handler(func=lambda m: storage.get_ps(m.from_user.id)=="city_search"
def handle_city(message):
    uid=message.from_user.id
    storage.set_city(uid,message.text.strip()); storage.clr_ps(uid)
    storage.set_interest(uid,None); storage.add_queue(uid,vip=True)
    send(uid,f"⏳ Поиск VIP из <b>{message.text.strip()}</b>…",markup=kb_leave())
    do_match(uid,vip=True)

@bot.message_handler(func=lambda m: m.text=="⏩ Без фильтра")
def btn_vip_nofilt(message):
    uid=message.from_user.id
    if not storage.is_vip(uid): return
    storage.set_interest(uid,None); storage.set_city(uid,None)
    storage.add_queue(uid,vip=True)
    send(uid,"⏳ Поиск VIP…",markup=kb_leave()); do_match(uid,vip=True)

@bot.message_handler(func=lambda m: m.text=="⏹️ Покинуть очередь")
def btn_leave_q(message):
    uid=message.from_user.id
    storage.leave_queue(uid); send(uid,"✅ Вы покинули очередь.",markup=kb_main(u

@bot.message_handler(func=lambda m: m.text=="❌ Завершить диалог")
def btn_end(message):
    uid=message.from_user.id; partner=storage.end_chat(uid)
    send(uid,"🔚 Диалог завершён.",markup=kb_main(uid))
    if partner:
        if storage.is_vip(partner) and storage.is_auto_next(partner):
            send(partner,"❌ Собеседник ушёл. Ищу следующего…",markup=kb_leave())
            storage.set_interest(partner,None); storage.set_city(partner,None)
            storage.add_queue(partner,vip=True); do_match(partner,vip=True)
        else: partner_left_msg(partner)

@bot.message_handler(func=lambda m: m.text=="🚩 Пожаловаться")
def btn_report(message):
    uid=message.from_user.id
    if not storage.is_in_chat(uid): send(uid,"⚠️ Вы не в чате.",markup=kb_main(ui
    send(uid,"🚩 <b>Выберите причину жалобы:</b>",markup=kb_report())

@bot.message_handler(func=lambda m: m.text in REPORT_REASONS)
def handle_report(message):
    uid=message.from_user.id; partner=storage.get_partner(uid)
    if not partner: send(uid,"⚠️ Вы не в чате.",markup=kb_main(uid)); return
    cnt=storage.add_report(partner,uid,message.text)
    for adm in ADMIN_IDS:
```

```
        send(adm,f"🚩 <b>Жалоба</b>\nНа: <code>{partner}</code>\nПричина: {messag
    if cnt>=3:
        storage.ban(partner,f"Автобан: {cnt} жалобы",days=0)
        send(partner,"🚫 Ваш аккаунт заблокирован из-за жалоб.")
    storage.end_chat(uid); send(uid,"✅ Жалоба отправлена.",markup=kb_main(uid))
    if partner and not storage.is_banned(partner): partner_left_msg(partner)

@bot.message_handler(func=lambda m: m.text=="↩️ Отмена")
def btn_cancel(message):
    uid=message.from_user.id; storage.clr_as(uid)
    send(uid,"Отменено.",markup=kb_chat(uid) if storage.is_in_chat(uid) else kb_m

# ── Кошелёк ──────────────────────────────────────────────────

@bot.message_handler(func=lambda m: m.text=="💰 Кошелёк")
def btn_wallet(message):
    uid=message.from_user.id; nayms=storage.get_nayms(uid)
    send(uid,
         f"💰 <b>Ваш кошелёк</b>\n\n⚡ Nayms: <b>{nayms}</b>\n\n"
         f"Nayms — валюта NoneName Chat.\nИспользуйте для покупки и отправки пода
         f"<b>Пополнить баланс:</b>", markup=kb_wallet())

@bot.callback_query_handler(func=lambda c: c.data.startswith("buy_nayms_"))
def cb_buy_nayms(call):
    uid=call.from_user.id
    parts=call.data.split("_")
    stars=int(parts[2]); nayms=int(parts[3])
    try: bot.answer_callback_query(call.id)
    except: pass
    bot.send_invoice(uid,
        title=f"Купить {nayms} ⚡ Nayms",
        description=f"Пополнение баланса на {nayms} ⚡.",
        invoice_payload=f"nayms_{nayms}", provider_token="", currency="XTR",
        prices=[types.LabeledPrice(label=f"{nayms} Nayms", amount=stars)])

# ── Магазин v2.2 (с подтверждением) ──────────────────────────

@bot.message_handler(func=lambda m: m.text=="🛍️ Магазин")
def btn_shop(message):
    uid=message.from_user.id; nayms=storage.get_nayms(uid)
    send(uid,
         f"🛍️ <b>Магазин NoneName Chat</b>\n\n"
         f"Ваш баланс: <b>{nayms} ⚡</b>\n\n"
         f"💝 <b>Сердце</b> — {HEART_PRICE} ⚡\n"
         f"Красивая анимация-подарок. Отправьте собеседнику или другу!",
         markup=kb_shop(uid))
```

```
@bot.callback_query_handler(func=lambda c: c.data=="buy_item_heart")
def cb_buy_heart(call):
    uid=call.from_user.id; nayms=storage.get_nayms(uid)
    try: bot.answer_callback_query(call.id)
    except: pass
    if not storage.has_nayms(uid, HEART_PRICE):
        send(uid,f"❌ Недостаточно Nayms!\nНужно: <b>{HEART_PRICE} ⚡</b>\nУ вас:
    # Подтверждение покупки
    kb = types.InlineKeyboardMarkup(row_width=2)
    kb.add(
        types.InlineKeyboardButton("✅ Да, купить",  callback_data="confirm_buy_h
        types.InlineKeyboardButton("❌ Нет, отмена", callback_data="cancel_action
    )
    send(uid,
         f"💝 <b>Подтверждение покупки</b>\n\n"
         f"Купить <b>Сердце</b> за <b>{HEART_PRICE} ⚡</b>?\n"
         f"Ваш баланс: <b>{nayms} ⚡</b>\n"
         f"После покупки: <b>{nayms-HEART_PRICE} ⚡</b>",
         markup=kb)

@bot.callback_query_handler(func=lambda c: c.data=="confirm_buy_heart")
def cb_confirm_buy_heart(call):
    uid=call.from_user.id
    try: bot.answer_callback_query(call.id)
    except: pass
    if not storage.has_nayms(uid, HEART_PRICE):
        send(uid,"❌ Недостаточно Nayms!"); return
    storage.add_nayms(uid,-HEART_PRICE); storage.add_item(uid,"heart",is_purchase
    bal=storage.get_nayms(uid)
    send(uid,f"✅ <b>Сердце куплено!</b>\n\n💝 Оно в «🎁 Мои подарки».\nОстаток:

@bot.callback_query_handler(func=lambda c: c.data=="cancel_action")
def cb_cancel_action(call):
    try: bot.answer_callback_query(call.id,"Отменено.")
    except: pass
    send(call.from_user.id,"❌ Действие отменено.")

# ── Мои подарки v2.2 (с подтверждением продажи) ──────────────

@bot.message_handler(func=lambda m: m.text=="🎁 Мои подарки")
def btn_my_gifts(message):
    uid=message.from_user.id; items=storage.get_inventory(uid)
    if not items:
        send(uid,"🎁 <b>Мои подарки</b>\n\nУ вас пока нет предметов. Загляните в
             markup=kb_main(uid)); return
    kb=types.InlineKeyboardMarkup(row_width=1)
    now=datetime.now(); lines=["🎁 <b>Мои предметы:</b>\n"]
```

```
    for item in items:
        iid=item["id"]; name=item["item_name"]
        received=item["received_at"]; purchased=item["is_purchased"]
        label="💝 Сердце" if name=="heart" else name
        try: rdt=datetime.strptime(received,"%Y-%m-%d %H:%M:%S")
        except: rdt=now
        days_ago=(now-rdt).days; date_str=rdt.strftime("%d.%m.%Y")
        sell_price=HEART_SELL_OWN if purchased else HEART_SELL_GIFT
        if purchased: source="🛒 куплено"; can_sell=True
        else:         source="🎁 подарок"; can_sell=(days_ago<7)
        lines.append(f"• {label} | {source} | {date_str}")
        if can_sell:
            kb.add(types.InlineKeyboardButton(
                f"Продать {label} за {sell_price} ⚡ (ID:{iid})",
                callback_data=f"sell_item_{iid}"))
        elif not purchased:
            lines[-1]+=" | ⏳ срок истёк"
    send(uid,"\n".join(lines),markup=kb)

@bot.callback_query_handler(func=lambda c: c.data.startswith("sell_item_"))
def cb_sell_item(call):
    uid=call.from_user.id; item_id=int(call.data.split("_")[-1])
    try: bot.answer_callback_query(call.id)
    except: pass
    item=storage.get_item(item_id,uid)
    if not item: send(uid,"❌ Предмет не найден."); return
    purchased=item["is_purchased"]
    try: rdt=datetime.strptime(item["received_at"],"%Y-%m-%d %H:%M:%S")
    except: rdt=datetime.now()
    days_ago=(datetime.now()-rdt).days
    if not purchased and days_ago>=7: send(uid,"❌ Срок продажи истёк (7 дней).")
    sell_price=HEART_SELL_OWN if purchased else HEART_SELL_GIFT
    # Подтверждение продажи
    kb=types.InlineKeyboardMarkup(row_width=2)
    kb.add(
        types.InlineKeyboardButton("✅ Да, продать",  callback_data=f"confirm_sel
        types.InlineKeyboardButton("❌ Нет, отмена",  callback_data="cancel_actic
    )
    src="куплено вами" if purchased else f"получено в подарок ({days_ago} дн. наз
    send(uid,
         f"💝 <b>Подтверждение продажи</b>\n\n"
         f"Продать <b>Сердце</b> за <b>{sell_price} ⚡</b>?\n"
         f"Предмет: {src}", markup=kb)

@bot.callback_query_handler(func=lambda c: c.data.startswith("confirm_sell_"))
def cb_confirm_sell(call):
    uid=call.from_user.id; item_id=int(call.data.split("_")[-1])
```

```
    try: bot.answer_callback_query(call.id)
    except: pass
    item=storage.get_item(item_id,uid)
    if not item: send(uid,"❌ Предмет не найден."); return
    purchased=item["is_purchased"]
    try: rdt=datetime.strptime(item["received_at"],"%Y-%m-%d %H:%M:%S")
    except: rdt=datetime.now()
    if not purchased and (datetime.now()-rdt).days>=7:
        send(uid,"❌ Срок продажи истёк."); return
    sell_price=HEART_SELL_OWN if purchased else HEART_SELL_GIFT
    storage.remove_item(item_id); storage.add_nayms(uid,sell_price)
    send(uid,f"✅ Продано за <b>{sell_price} ⚡</b>!\nВаш баланс: <b>{storage.get

# ── Подарок в чате ──────────────────────────────────────────

@bot.message_handler(func=lambda m: m.text=="🎁 Подарок")
def btn_gift_in_chat(message):
    uid=message.from_user.id
    if not storage.is_in_chat(uid): send(uid,"⚠️ Вы не в чате.",markup=kb_main(ui
    hearts=storage.count_item(uid,"heart")
    if hearts==0:
        send(uid,f"🎁 У вас нет подарков.\n\nКупите в «🛍️ Магазин» за {HEART_PRIC
    send(uid,f"🎁 <b>Выберите подарок</b>\nДоступно сердец: <b>{hearts}</b>",mark

@bot.callback_query_handler(func=lambda c: c.data=="gift_heart")
def cb_send_heart_chat(call):
    uid=call.from_user.id; partner=storage.get_partner(uid)
    try: bot.answer_callback_query(call.id)
    except: pass
    if not partner: send(uid,"⚠️ Вы не в чате."); return
    items=storage.get_inventory(uid)
    heart_item=next((i for i in items if i["item_name"]=="heart"),None)
    if not heart_item: send(uid,"❌ У вас нет сердец."); return
    storage.set_as(uid,{"action":"pgift","target":partner,"step":"ask_note","in_c
    kb=types.ReplyKeyboardMarkup(resize_keyboard=True,row_width=2)
    kb.add(types.KeyboardButton("✅ Да, добавить записку"),types.KeyboardButton("
    send(uid,"💝 Хотите добавить записку к подарку?",markup=kb)

# ── Поиск людей ──────────────────────────────────────────

@bot.message_handler(func=lambda m: m.text=="🔎 Поиск людей")
def btn_search_people(message):
    uid=message.from_user.id
    send(uid,"🔎 <b>Поиск пользователей</b>\n\nВведите @username или числовой ID:
    storage.set_ps(uid,"search_user")

@bot.message_handler(func=lambda m: storage.get_ps(m.from_user.id)=="search_user"
```

```
def handle_search_user(message):
    uid=message.from_user.id; query=message.text.strip()
    storage.clr_ps(uid)
    results=storage.search_users(query)
    if not results: send(uid,"❌ Пользователей не найдено.",markup=kb_main(uid));
    kb=types.InlineKeyboardMarkup(row_width=1)
    for r in results:
        target_uid=r["uid"]; uname=r["username"]
        label=f"@{uname}" if uname else f"ID: {target_uid}"
        kb.add(types.InlineKeyboardButton(label,callback_data=f"view_profile_{tar
    send(uid,f"🔎 Найдено: <b>{len(results)}</b> пользователей:",markup=kb)

@bot.callback_query_handler(func=lambda c: c.data.startswith("view_profile_"))
def cb_view_profile(call):
    uid=call.from_user.id; target=int(call.data.split("_")[-1])
    try: bot.answer_callback_query(call.id)
    except: pass
    u=storage.get_user(target)
    if not u: send(uid,"❌ Пользователь не найден."); return
    uname=u["username"]; chats=u["chats"]
    rank=get_rank(chats); nr,needed=get_next_rank(chats)
    progress=f"До следующего ранга: ещё <b>{needed}</b> диалогов" if nr else "🏆
    vip_mark=" 👑" if storage.is_vip(target) else ""
    items=storage.get_inventory(target)
    hearts=sum(1 for i in items if i["item_name"]=="heart")
    lines=[
        f"👤 <b>Профиль {'@'+uname if uname else str(target)}</b>{vip_mark}\n",
        f"🎖️ Ранг: <b>{rank}</b>",
        f"📈 {progress}",
        f"💬 Диалогов: {chats}",
        f"💝 Подарков в коллекции: {hearts}",
    ]
    profile=storage.get_profile(target)
    if profile and any([profile["city"],profile["age"],profile["gender"],profile[
        lines.append("\n"+fmt_profile(profile,public=True))
    send(uid,"\n".join(lines),markup=kb_public_profile(target))

@bot.callback_query_handler(func=lambda c: c.data.startswith("pgift_"))
def cb_profile_gift(call):
    uid=call.from_user.id; target=int(call.data.split("_")[-1])
    try: bot.answer_callback_query(call.id)
    except: pass
    hearts=storage.count_item(uid,"heart")
    if hearts==0:
        send(uid,f"💝 У вас нет сердец.\n\nКупите в «🛍️ Магазин» за {HEART_PRICE}
    storage.set_as(uid,{"action":"pgift","target":target,"step":"ask_note"})
    kb=types.ReplyKeyboardMarkup(resize_keyboard=True,row_width=2)
```

```
    kb.add(types.KeyboardButton("✅ Да, добавить записку"),types.KeyboardButton("
    send(uid,"💝 Хотите добавить записку к подарку?",markup=kb)

@bot.callback_query_handler(func=lambda c: c.data.startswith("pdm_"))
def cb_profile_dm(call):
    uid=call.from_user.id; target=int(call.data.split("_")[-1])
    try: bot.answer_callback_query(call.id)
    except: pass
    if uid==target: send(uid,"❌ Нельзя написать самому себе."); return
    if not storage.can_send_dm(uid):
        send(uid,f"⏱ Вы исчерпали лимит ({DM_LIMIT_DAY} сообщений в день).\n👑 Р
    uname=storage.get_username(target)
    to_str=f"@{uname}" if uname else str(target)
    if storage.is_vip(uid):
        storage.set_as(uid,{"action":"dm","target":target,"step":"ask_anon"})
        kb=types.ReplyKeyboardMarkup(resize_keyboard=True,row_width=2)
        kb.add(types.KeyboardButton("🕶 Анонимно"),types.KeyboardButton("📨 Под с
        send(uid,f"📩 Написать {to_str}.\nКак отправить?",markup=kb)
    else:
        storage.set_as(uid,{"action":"dm","target":target,"step":"get_text","is_a
        send(uid,f"📩 Введите сообщение для {to_str}:\n<i>(лимит: {storage.dm_lef

# ── Wizard (DM + Gift) ──────────────────────────────────────

@bot.message_handler(func=lambda m: m.text in ["✅ Да, добавить записку","⏩ Нет,
                                               "🕶 Анонимно","📨 Под своим нико
def handle_wizard_choice(message):
    uid=message.from_user.id; state=storage.get_as(uid)
    if not state: return
    action=state.get("action"); step=state.get("step"); text=message.text
    if action=="pgift" and step=="ask_note":
        if text=="✅ Да, добавить записку":
            state["step"]="get_note"; storage.set_as(uid,state)
            send(uid,"✍️ Введите текст записки:")
        else:
            _send_heart_to(uid,state["target"],note="")
        return
    if action=="dm" and step=="ask_anon":
        is_anon=1 if text=="🕶 Анонимно" else 0
        state["is_anon"]=is_anon; state["step"]="get_text"; storage.set_as(uid,st
        target=state["target"]; uname=storage.get_username(target)
        to_str=f"@{uname}" if uname else str(target)
        send(uid,f"📩 Введите сообщение для {to_str}:\n"
                 f"<i>{'Анонимно' if is_anon else 'Под вашим ником'} | лимит: {'∞
        return

@bot.message_handler(func=lambda m: storage.get_as(m.from_user.id) is not None
```

```
                    and storage.get_as(m.from_user.id).get("action") in ("pgift"
                    and not is_admin(m.from_user.id))
def handle_wizard_text(message):
    uid=message.from_user.id; state=storage.get_as(uid)
    if not state: return
    action=state.get("action"); step=state.get("step"); text=message.text.strip()
    if action=="pgift" and step=="get_note":
        _send_heart_to(uid,state["target"],note=text); return
    if action=="dm" and step=="get_text":
        target=state["target"]; is_anon=state.get("is_anon",0)
        if not storage.can_send_dm(uid):
            send(uid,f"⏱️ Лимит исчерпан ({DM_LIMIT_DAY}/день). VIP — безлимит!",
            storage.clr_as(uid); return
        storage.save_dm(uid,target,text,is_anon)
        storage.record_dm_sent(uid)
        storage.clr_as(uid)
        from_str="Анонимный пользователь" if (is_anon or not storage.get_username
        send(target,
             f"📩 <b>Вам новое сообщение!</b>\n"
             f"От: <b>{from_str}</b>\n\n{text}", markup=kb_main(target))
        send(uid,"✅ Сообщение доставлено!",markup=kb_main(uid))
        return

def _send_heart_to(uid:int, target:int, note:str=""):
    items=storage.get_inventory(uid)
    heart_item=next((i for i in items if i["item_name"]=="heart"),None)
    if not heart_item:
        send(uid,"❌ У вас нет сердец.",markup=kb_main(uid))
        storage.clr_as(uid); return
    storage.remove_item(heart_item["id"])
    storage.add_item(target,"heart",is_purchased=0)
    uname=storage.get_username(uid) or str(uid)
    caption=(f"💝 <b>Вам подарили Сердце!</b>\nОт: @{uname}\n\n{note}" if note
             else f"💝 <b>Вам подарили Сердце!</b>\nОт: @{uname}")
    send_heart_animation(target, caption)
    send(uid,"✅ <b>Сердце отправлено!</b> 💝",markup=kb_main(uid))
    storage.clr_as(uid)

# ── VIP покупка ──────────────────────────────────────────────

@bot.message_handler(func=lambda m: m.text=="⭐ Купить VIP")
def btn_buy_vip(message):
    uid=message.from_user.id; exp=storage.vip_expires(uid)
    prefix=f"👑 У вас VIP до <b>{exp}</b>. Продлить:\n\n" if exp else ""
    send(uid, prefix+
         "👑 <b>VIP-статус открывает:</b>\n\n"
         "• 👑 Значок VIP собеседнику\n• 💬 Отдельный VIP-чат\n"
```

```
        "• 🌍 Поиск по городу\n• 📋 Личная анкета\n"
        "• 📜 История диалогов\n• 🔄 Авто-следующий\n"
        "• 📎 Медиа без ограничений\n• 📩 Безлимитные DM\n"
        "• 👤 Анонимный режим в DM\n\nВыберите план:", markup=kb_vip_buy())

@bot.callback_query_handler(func=lambda c: c.data in ["buy_vip_7","buy_vip_30"])
def cb_buy_vip(call):
    uid=call.from_user.id; days=7 if call.data=="buy_vip_7" else 30; stars=5 if d
    try: bot.answer_callback_query(call.id)
    except: pass
    bot.send_invoice(uid, title=f"VIP на {days} дней 👑",
        description=f"VIP-статус на {days} дней.",
        invoice_payload=f"vip_{days}", provider_token="", currency="XTR",
        prices=[types.LabeledPrice(label=f"VIP {days} дней", amount=stars)])

@bot.pre_checkout_query_handler(func=lambda q: True)
def pre_checkout(query): bot.answer_pre_checkout_query(query.id, ok=True)

@bot.message_handler(content_types=["successful_payment"])
def successful_payment(message):
    uid=message.from_user.id; payload=message.successful_payment.invoice_payload
    if payload.startswith("vip_"):
        days=int(payload.split("_")[1]); storage.give_vip(uid,days); exp=storage.
        send(uid,f"🎉 <b>VIP на {days} дней!</b>\n👑 До: {exp}",markup=kb_main(ui
    elif payload.startswith("nayms_"):
        nayms=int(payload.split("_")[1]); storage.add_nayms(uid,nayms); bal=stora
        send(uid,f"⚡ <b>+{nayms} Nayms!</b>\nБаланс: <b>{bal} ⚡</b>",markup=kb_

# ── VIP Анкета ──────────────────────────────────────────────

@bot.message_handler(func=lambda m: m.text=="📋 Моя анкета")
def btn_my_profile(message):
    uid=message.from_user.id
    if not storage.is_vip(uid): send(uid,"🚫 Анкета — только VIP."); return
    p=storage.get_profile(uid)
    kb=types.InlineKeyboardMarkup()
    kb.add(types.InlineKeyboardButton("✏️ Создать / изменить",callback_data="edit
           types.InlineKeyboardButton("🗑️ Удалить",callback_data="del_profile"))
    send(uid, fmt_profile(p) if p else "У вас нет анкеты. Создайте:", markup=kb)

@bot.callback_query_handler(func=lambda c: c.data=="edit_profile")
def cb_edit_profile(call):
    uid=call.from_user.id
    try: bot.answer_callback_query(call.id)
    except: pass
    if not storage.is_vip(uid): return
    storage.set_ps(uid,"profile_city"); storage._tmp_profile[uid]={}
```

```
    send(uid,"📋 <b>Создание анкеты</b>\n\n"+PROFILE_Q["city"])

@bot.callback_query_handler(func=lambda c: c.data=="del_profile")
def cb_del_profile(call):
    uid=call.from_user.id
    try: bot.answer_callback_query(call.id)
    except: pass
    storage.del_profile(uid); send(uid,"✅ Анкета удалена.")

@bot.message_handler(func=lambda m: (storage.get_ps(m.from_user.id) or "").starts
def handle_profile_step(message):
    uid=message.from_user.id; step=storage.get_ps(uid).replace("profile_","")
    storage._tmp_profile.setdefault(uid,{})[step]=message.text.strip()
    idx=PROFILE_STEPS.index(step)
    if idx+1<len(PROFILE_STEPS):
        nxt=PROFILE_STEPS[idx+1]; storage.set_ps(uid,f"profile_{nxt}"); send(uid,
    else:
        profile=storage._tmp_profile.pop(uid,{})
        storage.set_profile(uid,profile); storage.clr_ps(uid)
        send(uid,"✅ <b>Анкета сохранена!</b>\n\n"+fmt_profile(storage.get_profil
             markup=kb_main(uid))

@bot.message_handler(func=lambda m: m.text=="👤 Анкета собеседника")
def btn_partner_profile(message):
    uid=message.from_user.id; partner=storage.get_partner(uid)
    if not partner: send(uid,"⚠️ Вы не в чате."); return
    send(uid,fmt_profile(storage.get_profile(partner)))

@bot.message_handler(func=lambda m: m.text=="📜 История чатов")
def btn_history(message):
    uid=message.from_user.id; partner=storage.get_partner(uid)
    if not storage.is_vip(uid): send(uid,"🚫 История — только VIP."); return
    if not partner: send(uid,"ℹ️ История отображается во время диалога."); return
    hist=storage.get_history(uid,partner)
    if not hist: send(uid,"📜 История пуста."); return
    lines=["📜 <b>История диалога:</b>\n"]
    for m in reversed(list(hist)): lines.append(f"[{m['created_at']}] <b>{m['send
    send(uid,"\n".join(lines))

# ═══════════════════════════════════════════
# 🔧 ADMIN ПАНЕЛЬ v2.2
# ═══════════════════════════════════════════

@bot.message_handler(func=lambda m: is_admin(m.from_user.id) and m.text==="🔙 Выйт
def adm_exit(message):
    storage.clr_as(message.from_user.id)
    send(message.from_user.id,"Вышли из панели.",markup=kb_main(message.from_user
```

```
@bot.message_handler(func=lambda m: is_admin(m.from_user.id) and m.text=="📊 Стат
def adm_stats(message):
    uid=message.from_user.id; users=storage.all_users(); vips=[u for u in users i
    send(uid,"⏳ Считаю активных...")
    active=count_active_users()
    send(uid,
         f"📊 <b>Статистика:</b>\n\n"
         f"👥 Всего: <b>{len(users)}</b>\n"
         f"✅ Активных: <b>{active}</b>\n"
         f"🚫 Заблокировали бота: <b>{len(users)-active}</b>\n"
         f"👑 VIP: <b>{len(vips)}</b>\n"
         f"💬 Чатов сейчас: <b>{len(storage._chats)//2}</b>\n"
         f"⏳ В очереди: <b>{len(storage._queue)+len(storage._vip_queue)}</b>",
         markup=kb_admin(uid))

@bot.message_handler(func=lambda m: is_admin(m.from_user.id) and m.text=="📋 Жалс
def adm_reports(message):
    uid=message.from_user.id; rows=storage.all_reported_users()
    if not rows: send(uid,"📋 Жалоб нет.",markup=kb_admin(uid)); return
    lines=["📋 <b>Пользователи с жалобами:</b>\n"]
    for r in rows:
        uname=storage.get_username(r["target_id"])
        u_str=f" (@{uname})" if uname else ""
        lines.append(f"🆔 <code>{r['target_id']}</code>{u_str} — жалоб: <b>{r['cn
        for rep in storage.get_reports(r["target_id"])[:3]:
            lines.append(f"   • {rep['reason']} ({rep['created_at']})")
    send(uid,"\n".join(lines),markup=kb_admin(uid))

@bot.message_handler(func=lambda m: is_admin(m.from_user.id) and m.text=="🔗 Рефе
def adm_refs(message):
    uid=message.from_user.id; rows=storage.all_refs()
    if not rows: send(uid,"🔗 Рефералов нет.",markup=kb_admin(uid)); return
    lines=["🔗 <b>Рефералы:</b>\n"]
    for r in rows:
        uname=storage.get_username(r["inviter_id"])
        u_str=f" (@{uname})" if uname else ""
        lines.append(f"🆔 <code>{r['inviter_id']}</code>{u_str} → <b>{r['cnt']}</
    send(uid,"\n".join(lines),markup=kb_admin(uid))

@bot.message_handler(func=lambda m: is_admin(m.from_user.id) and
                     m.text in ["🔄 Скрыть статус","🔄 Показать статус"])
def adm_toggle_incognito(message):
    uid=message.from_user.id
    state=storage.toggle_incognito(uid)
    if state:
        send(uid,"🕶️ Режим инкогнито <b>включён</b>.\nВаш статус скрыт в чатах.",
```

```
            markup=kb_admin(uid))
    else:
        send(uid,"🛡️ Статус <b>виден</b> собеседникам в чатах.",
            markup=kb_admin(uid))

@bot.message_handler(func=lambda m: is_admin(m.from_user.id) and m.text=="📣 Расс
def adm_broadcast_start(message):
    uid=message.from_user.id
    send(uid,"Введите текст рассылки:"); storage.set_as(uid,{"action":"broadcast"

@bot.message_handler(func=lambda m: is_admin(m.from_user.id) and m.text=="✉️ Напи
def adm_msg_user_start(message):
    uid=message.from_user.id
    send(uid,"Введите <b>Telegram ID</b> или <b>@username</b> пользователя:")
    storage.set_as(uid,{"action":"msg_user","step":"get_id"})

@bot.message_handler(func=lambda m: is_admin(m.from_user.id) and
                    m.text in ["🔨 Забанить","✅ Разбанить","👑 Выдать VIP","❌
def adm_action(message):
    uid=message.from_user.id
    act={"🔨 Забанить":"ban","✅ Разбанить":"unban",
         "👑 Выдать VIP":"give_vip","❌ Снять VIP":"remove_vip"}[message.text]
    storage.set_as(uid,{"action":act,"step":"get_id"})
    send(uid,"Введите <b>Telegram ID</b> или <b>@username</b> пользователя:")

@bot.message_handler(func=lambda m: is_admin(m.from_user.id) and storage.get_as(m
def adm_wizard(message):
    uid=message.from_user.id; state=storage.get_as(uid)
    if not state: return
    action=state.get("action"); step=state.get("step"); text=message.text.strip()

    if action=="broadcast":
        ok=fail=0
        for u in storage.all_users():
            try: bot.send_message(u,f"📣 <b>Сообщение от администратора:</b>\n\n{
            except: fail+=1
        storage.clr_as(uid)
        send(uid,f"✅ Доставлено: {ok}, не доставлено: {fail}.",markup=kb_admin(u

    if action=="msg_user":
        if step=="get_id":
            target=resolve_target(text)
            if not target: send(uid,"❌ Пользователь не найден. Введите ID или @u
            state["target"]=target; state["step"]="get_days"; storage.set_as(uid,
            uname=storage.get_username(target)
            send(uid,f"На сколько дней VIP для <code>{target}</code>{' (@'+uname+
        if step=="get_days":
```

```
        try: days=int(text)
        except: send(uid,"❌ Введите число."); return
        state["vip_days"]=days; state["step"]="get_text"; storage.set_as(uid,
        kb=types.ReplyKeyboardMarkup(resize_keyboard=True)
        kb.add(types.KeyboardButton("⏩ Пропустить сообщение"))
        send(uid,"Введите <b>текст сообщения</b> или нажмите «Пропустить»:",m
    if step=="get_text":
        target=state["target"]; vip_days=state.get("vip_days",0)
        skip=text=="⏩ Пропустить сообщение"
        if vip_days>0:
            storage.give_vip(target,vip_days); exp=storage.vip_expires(target
            send(target,f"🎉 <b>Вам выдан VIP статус администратором!</b>\n👑
        if not skip:
            try: bot.send_message(target,f"📩 <b>Сообщение от администратора:
            except Exception as e: st=f"❌ {e}"
        else: st="⏩ Пропущено."
        storage.clr_as(uid)
        uname=storage.get_username(target)
        res=[f"✅ <code>{target}</code>{' (@'+uname+')' if uname else ''}"]
        if vip_days>0: res.append(f"👑 VIP: {vip_days} дней.")
        res.append(st)
        send(uid,"\n".join(res),markup=kb_admin(uid)); return

if step=="get_id":
    target=resolve_target(text)
    if not target: send(uid,"❌ Пользователь не найден. Введите ID или @usern
    state["target"]=target
    uname=storage.get_username(target)
    u_str=f" (@{uname})" if uname else ""
    if action=="ban":
        state["step"]="get_reason"; storage.set_as(uid,state)
        send(uid,f"Введите <b>причину блокировки</b> для <code>{target}</code
    elif action=="give_vip":
        state["step"]="get_days"; storage.set_as(uid,state)
        send(uid,f"На сколько <b>дней</b> VIP для <code>{target}</code>{u_str
    elif action=="unban":
        storage.unban(target); storage.clr_as(uid)
        send(uid,f"✅ {target}{u_str} разблокирован.",markup=kb_admin(uid))
        send(target,"✅ Ваш аккаунт разблокирован. Добро пожаловать обратно!"
    elif action=="remove_vip":
        storage.remove_vip(target); storage.clr_as(uid)
        send(uid,f"✅ VIP у {target}{u_str} снят.",markup=kb_admin(uid))
        send(target,"ℹ️ <b>Ваш VIP статус был снят администратором.</b>")
    return

if action=="ban" and step=="get_reason":
    state["reason"]=text; state["step"]="get_days"; storage.set_as(uid,state)
```

```
        send(uid,"На сколько дней? (<b>0</b> = навсегда):"); return

    if step=="get_days" and action in ("ban","give_vip"):
        try: days=int(text)
        except: send(uid,"❌ Введите число."); return
        target=state["target"]
        if action=="ban":
            reason=state.get("reason","Нарушение правил")
            partner=storage.ban(target,reason,days); storage.clr_as(uid)
            period=f"на {days} дней" if days>0 else "навсегда"
            send(uid,f"✅ {target} заблокирован {period}.",markup=kb_admin(uid))
            send(target,f"🚫 <b>Вы заблокированы</b> {period}.\n📋 Причина: {reas
            if partner: partner_left_msg(partner)
        elif action=="give_vip":
            storage.give_vip(target,days); storage.clr_as(uid); exp=storage.vip_e
            send(uid,f"✅ VIP выдан {target} на {days} дней.",markup=kb_admin(uid
            send(target,f"🎉 <b>Вам выдан VIP статус администратором!</b>\n👑 На

# ── Пересылка сообщений ──────────────────────────────────────

MEDIA_TYPES=["photo","video","audio","voice","video_note","sticker","document","a

@bot.message_handler(func=lambda m: True, content_types=["text"]+MEDIA_TYPES)
def relay(message):
    uid=message.from_user.id; ct=message.content_type
    if ct=="text" and message.text in ALL_MENU: return
    if storage.get_ps(uid) or storage.get_as(uid): return
    if storage.is_banned(uid):
        info=storage.ban_info(uid)
        send(uid,f"🚫 Вы заблокированы.\n📋 Причина: {info['ban_reason'] if info
    if storage.rate_limited(uid): send(uid,"⏱ Не так быстро!"); return
    storage.record_msg(uid)
    if not storage.is_in_chat(uid): send(uid,"ℹ️ Нажмите «Найти собеседника».",ma
    partner=storage.get_partner(uid)
    if not partner: storage.end_chat(uid); send(uid,"⚠️ Ошибка. Попробуйте снова.
    if ct in MEDIA_TYPES and ct!="sticker" and not storage.is_vip(uid):
        if storage.media_limited(uid): send(uid,f"⏱ Лимит: {MEDIA_PER_MIN} медиа
    if ct in ["voice","video_note"] and not storage.is_vip(uid):
        send(uid,"🎙 Голосовые и кружочки — только VIP.\n\nНажмите «⭐ Купить VIP
    try:
        fwd(message,partner)
        if storage.is_vip(uid) or storage.is_vip(partner):
            txt=message.text or message.caption or f"[{ct}]"
            storage.save_msg_history(uid,partner,uid,txt)
    except telebot.apihelper.ApiTelegramException as e:
        logger.warning(f"Пересылка {uid}->{partner}: {e}")
        storage.end_chat(uid); send(uid,"❌ Собеседник недоступен.",markup=kb_mai
```

```
# ═══════════════════════════════════════════════
#  🔄  ФОНОВЫЕ ЗАДАЧИ
# ═══════════════════════════════════════════════

def matchmaker():
    while True:
        try:
            for uid in storage.q_snap():
                if storage.in_queue(uid) and not storage.is_in_chat(uid): do_matc
            for uid in storage.vq_snap():
                if storage.in_vip_queue(uid) and not storage.is_in_chat(uid): do_
        except Exception as e: logger.error(f"Матчмейкер: {e}")
        time.sleep(2)

def vip_checker():
    notified=set()
    while True:
        try:
            rows=storage._fetch("SELECT uid,vip_until FROM users WHERE vip_until>
            now=time.time()
            for r in rows:
                uid=r["uid"]; left=r["vip_until"]-now
                if 0<left<86400 and uid not in notified:
                    notified.add(uid); h=int(left//3600)
                    send(uid,f"⏰ VIP истекает через <b>{h} ч.</b> Нажмите «⭐ Ку
                elif left<=0 and uid in notified: notified.discard(uid)
        except Exception as e: logger.error(f"VIP checker: {e}")
        time.sleep(600)

# ═══════════════════════════════════════════════
#  🚀  ЗАПУСК
# ═══════════════════════════════════════════════

if __name__ == "__main__":
    os.makedirs(IMAGES_DIR, exist_ok=True)
    logger.info("="*50)
    logger.info("  NoneName Chat v2.2 — Full Update")
    logger.info("="*50)
    threading.Thread(target=matchmaker,  daemon=True).start()
    threading.Thread(target=vip_checker, daemon=True).start()
    logger.info("Бот запущен!")
    bot.infinity_polling(timeout=30, long_polling_timeout=20)
```